

colab.research.google.com/drive/1eNzwLAurdwmYmPwfRMalbHdnt1IENeJa#scrollTo=asCrZvJowyzy

SimpleLR.ipynb

File Edit View Insert Runtime Tools Help All changes saved

Comment

Share

RAM

Disk

Gemini

Files

..

sample_data

Salary_Data.csv

Upload salary CSV file From your system

SIMPLE LINEAR REGRESSION

Importing the libraries

0s

import numpy as np

import matplotlib.pyplot as plt

import pandas as pd

completed at 6:20 PM

Files

sample_data

Salary_Data.csv

74.90 GB available

+ Code + Text

SIMPLE LINEAR REGRESSION

Importing the libraries

```
[1] import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

Importing the Salary Dataset

```
dataset = pd.read_csv('Salary_Data.csv')
x = dataset.iloc[:, :-1].values
y = dataset.iloc[:, -1].values
```



Files



sample_data



Salary_Data.csv



+ Code + Text

SIMPLE LINEAR REGRESSION

Importing the libraries

```
[1] import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

Importing the Salary Dataset

```
dataset = pd.read_csv('Salary_Data.csv')
x = dataset.iloc[:, :-1].values
y = dataset.iloc[:, -1].values
```

Splitting the dataset into Training and test set

```
[15] from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size= 0.2, random_state=0)
```

Training the simple regression model on the training set

Start coding or generate with AI.

Train and test data split

Salary_Data.csv

1 to 30 of 30 entries

Filter

YearsExperience	Salary
1.1	39343.00
1.3	46205.00
1.5	37731.00
2.0	43525.00
2.2	39891.00
2.9	56642.00
3.0	60150.00
3.2	54445.00
3.2	64445.00
3.7	57189.00
3.9	63218.00
4.0	55794.00
4.0	56957.00
4.1	57081.00
4.5	61111.00
4.9	67938.00
5.1	66029.00
5.3	83088.00
5.9	81363.00
6.0	93940.00
6.8	91738.00
7.1	98273.00
7.9	101302.00
8.2	113812.00

This line imports the **LinearRegression** class from the **sklearn.linear_model** module, which provides functionality for performing linear regression.

Importing the Salary Dataset

```
dataset = pd.read_csv('Salary_Data.csv')
x = dataset.iloc[:, :-1].values
y = dataset.iloc[:, -1].values
```

Splitting the dataset into Training and test set

```
[15] from sklearn.model_selection import train_test_split
      x_train, x_test, y_train, y_test = train_test_split(x, y, test_size= 0.2, random_state=0)
```

Training the simple regression model on the training set

```
[16] from sklearn.linear_model import LinearRegression
      regressor = LinearRegression()
      regressor.fit(x_train, y_train)
```

LinearRegression

Salary_Data.csv

1 to 30 of 30 entries		Filter	
YearsExperience		Salary	
1.1		39343.00	
1.3		46205.00	
1.5		37731.00	
2.0		43525.00	
2.2		39891.00	
2.9		56642.00	
3.0		60150.00	
3.2		54445.00	
3.2		64445.00	
3.7		57189.00	
3.9		63218.00	
4.0		55794.00	
4.0		56957.00	
4.1		57081.00	
4.5		61111.00	
4.9		67938.00	
5.1		66029.00	
5.3		83088.00	
5.9		81363.00	
6.0		93940.00	
6.8		91738.00	
7.1		98273.00	
7.9		101302.00	
8.2		113812.00	

colab.research.google.com/drive/1eNzwLAurdwmYmPwFRMalbHdnt1IEneJa#scrollTo=NSwKeBfQzEud

SimpleLR.ipynb

File Edit View Insert Runtime Tools Help All changes saved

Here, you're **creating an instance** of the **LinearRegression** class and storing it in the **variable regressor**.

Files

- sample_data
- Salary_Data.csv

Importing the Salary Dataset

```
dataset = pd.read_csv('Salary_Data.csv')
x = dataset.iloc[:, :-1].values
y = dataset.iloc[:, -1].values
```

Splitting the dataset into Training and test set

```
[15] from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size= 0.2, random_state=0)
```

Training the simple regression model on the training set

```
[16] from sklearn.linear_model import LinearRegression
regressor = LinearRegression()
regressor.fit(x_train, y_train)
```

LinearRegression

Salary_Data.csv

1 to 30 of 30 entries Filter

YearsExperience	Salary
1.1	39343.00
1.3	46205.00
1.5	37731.00
2.0	43525.00
2.2	39891.00
2.9	56642.00
3.0	60150.00
3.2	54445.00
3.2	64445.00
3.7	57189.00
3.9	63218.00
4.0	55794.00
4.0	56957.00
4.1	57081.00
4.5	61111.00
4.9	67938.00
5.1	66029.00
5.3	83088.00
5.9	81363.00
6.0	93940.00
6.8	91738.00
7.1	98273.00
7.9	101302.00
8.2	113812.00

74.90 GB available

completed at 7:01 PM

Files

- sample_data
- Salary_Data.csv

Importing the Salary Dataset

```
dataset = pd.read_csv('Salary_Data.csv')
x = dataset.iloc[:, :-1].values
y = dataset.iloc[:, -1].values
```

Splitting the dataset into Training and test set

```
[15] from sklearn.model_selection import train_test_split
      x_train, x_test, y_train, y_test = train_test_split(x, y, test_size= 0.2, random_state=0)
```

Training the simple regression model on the training set

```
[16] from sklearn.linear_model import LinearRegression
      regressor = LinearRegression()
      regressor.fit(x_train, y_train)
```

LinearRegression

Salary_Data.csv

1 to 30 of 30 entries

YearsExperience	Salary
1.1	39343.00
1.3	46205.00
1.5	37731.00
2.0	43525.00
2.2	39891.00
2.9	56642.00
3.0	60150.00
3.2	54445.00
3.2	64445.00
3.7	57189.00
3.9	63218.00
4.0	55794.00
4.0	56957.00
4.1	57081.00
4.5	61111.00
4.9	67938.00
5.1	66029.00
5.3	83088.00
5.9	81363.00
6.0	93940.00
6.8	91738.00
7.1	98273.00
7.9	101302.00
8.2	113812.00

This instance
(regressor)
represents the
linear regression
model

The **fit** method calculates the optimal values for the model's parameters (**slope and intercept**) based on the training data.

LinearRegression() **creates the model.**

fit(x_train, y_train) **trains the model** on the provided training data, allowing it to **learn the relationship between the input features (x_train) and the target variable (y_train).**

regressor.predict(x_test) uses the **trained linear regression model (regressor)** to **predict** the **target values** for the input features provided in **x_test**.

colab.research.google.com/drive/1eNzwLAurdwmYmPwfRMalbHdnt1IEneJa#scrollTo=1_KH0tZbA7Rx

SimpleLR.ipynb

File Edit View Insert Runtime Tools Help

Files

- ..
- sample_data
- Salary_Data.csv

Training the simple regression model on the training set

```
[16] from sklearn.linear_model import LinearRegression
      regressor = LinearRegression()
      regressor.fit(x_train,y_train)
```

LinearRegression

predict the test set

```
y_pred = regressor.predict(x_test)
```

The predicted values are stored in the variable **y_pred**.

You can then compare **y_pred** with the actual values (**y_test**) to evaluate the performance of your model.

74.90 GB available

Salary_Data.csv

1 to 30 of 30 entries Filter

YearsExperience	Salary
1.1	39343.00
1.3	46205.00
1.5	37731.00
2.0	43525.00
2.2	39891.00
2.9	56642.00
3.0	60150.00
3.2	54445.00
3.2	64445.00
3.7	57189.00
3.9	63218.00
4.0	55794.00
4.0	56957.00
4.1	57081.00
4.5	61111.00
4.9	67938.00
5.1	66029.00
5.3	83088.00
5.9	81363.00
6.0	93940.00
6.8	91738.00
7.1	98273.00
7.9	101302.00
8.2	113812.00

What Happens During predict?

The model applies the **learned linear relationship (slope and intercept)** from the **training phase** to each sample in **x_test** to predict the corresponding **y values**.

For each feature value in **x_test**, the model calculates the **predicted output** using the equation of the line (in case of **simple linear regression**):

$$\text{PREDICTED } y = \text{INTERCEPT} + (\text{SLOPE} \times x)$$

Files

- ..
- sample_data
- Salary_Data.csv

+ Code + Text

Visualizing the Test set result

```
plt.scatter(x_test,y_test,color = 'red')
plt.plot(x_test, y_pred ,color = 'blue')
plt.title('Salary vs Experience (Training set)')
plt.xlabel('years of experience')
plt.ylabel('salary')
plt.show()
```



plt.scatter creates a scatter plot.
x_test is plotted on the x-axis (years of experience).
y_test is plotted on the y-axis (salary).

This **red color plots** visualizes the actual **data points** from the **test set**, showing the **real relationship** between **years of experience** and **salary**.

colab.research.google.com/drive/1eNzWLAurdwmYmPwFRMalbHdnt1IEneJa#scrollTo=hUsf9bptOJsl

SimpleLR.ipynb

File Edit View Insert Runtime Tools Help All changes saved

Files

..

sample_data

Salary_Data.csv

+ Code + Text

Visualizing the Test set result

```
plt.scatter(x_test,y_test,color='red')
plt.plot(x_test,y_pred,color='blue')
plt.title('Salary vs Experience (Training set)')
plt.xlabel('years of experience')
plt.ylabel('salary')
plt.show()
```

Salary vs Experience (Training set)

years of experience	salary
1	37000
8	64000
9	57000
12	110000
16	123000

plt.plot is used to plot a line.
x_test is plotted on the x-axis.
y_pred is plotted on the y-axis.

y_pred contains the **predicted salary values** for the corresponding **x_test** values, generated by the model using **regressor.predict(x_test)**.

0s completed at 8:34 PM

74.90 GB available



Files

sample_data

Salary_Data.csv

+ Code + Text

years or experience

Making a single prediction (for example the salary of an employee with 12 years of experience)

```
[30] print(regressor.predict([[12]]))
```

```
[138531.00067138]
```

Therefore, our model predicts that the salary of an employee with 12 years of experience is** \$ 138967,5.**

Important note: Notice that the value of the feature (**12 years**) was input in a double pair of square brackets. That's because the "predict" method always expects a **2D array** as the format of its inputs. And putting 12 into a double pair of square brackets makes the input exactly a 2D array. Simply put:

12→scalar

[12]→1D array

[[12]]→2D array

Double-click (or enter) to edit

Getting the final linear regression equation with the values of the coefficients

RAM Disk Gemini

Salary_Data.csv

2.9	56642.00
3.0	60150.00
3.2	54445.00
3.2	64445.00
3.7	57189.00
3.9	63218.00
4.0	55794.00
4.0	56957.00
4.1	57081.00
4.5	61111.00
4.9	67938.00
5.1	66029.00
5.3	83088.00
5.9	81363.00
6.0	93940.00
6.8	91738.00
7.1	98273.00
7.9	101302.00
8.2	113812.00
8.7	109431.00
9.0	105582.00
9.5	116969.00
9.6	112635.00
10.3	122391.00
10.5	121872.00

Show 50 per page

Intercept: 26780.0091506 means that if a person has 0 years of experience, the predicted starting salary (based on this model) would be approximately \$26816.

CO

SimpleLR.ipynb

☆

File Edit View Insert Runtime Tools Help All changes saved

Files

sample_data

Salary_Data.csv

+ Code + Text

Double-click (or enter) to edit

Getting the final linear regression equation with the values of the coefficients

[31] print(regressor.coef_)
print(regressor.intercept_)

[9312.57512673]
26780.00915062818

Double-click (or enter) to edit

Therefore, the equation of our simple linear regression model is:

Salary = 9345.94 × YearsExperience + 26816.19

Important Note: To get these coefficients we called the "coef_" and "intercept_" attributes from our regressor object. Attributes in Python are different than methods and usually return a simple value or an array of values.

RAM
Disk

Gemini

Salary_Data.csv

2.9	56642.00
3.0	60150.00
3.2	54445.00
3.2	64445.00
3.7	57189.00
3.9	63218.00
4.0	55794.00
4.0	56957.00
4.1	57081.00
4.5	61111.00
4.9	67938.00
5.1	66029.00
5.3	83088.00
5.9	81363.00
6.0	93940.00
6.8	91738.00
7.1	98273.00
7.9	101302.00
8.2	113812.00
8.7	109431.00
9.0	105582.00
9.5	116969.00
9.6	112635.00
10.3	122391.00
10.5	121872.00

Show 50 per page

0s completed at 9:11 PM

Coefficient: 9345.94 suggests that for every additional year of experience ,the model predicts an increase in salary by \$935.94, assuming a linear relationship.

Prediction for 11.5 years

✓
0s

▶ `print(regressor.predict([[11.5]]))`

↔ `[133874.71310801]`

Prediction for 0 years of experience

✓
0s

▶ `print(regressor.predict([[0]]))`

↔ `[26780.09915063]`